

Release Management or Erratic Change?

Maintaining distributed, Web and n-tier applications accounts for most of their growing life cycle costs. Establishing release management practices can control costs, reduce disruption and speed delivery.

Core Topics

Applications Development: Applications Development Project Management; Applications Development Software Quality and Management

Key Issues

What strategies, technologies and vendors will enable AD organizations to provide effective software change management?

What project management best practices will assist AD organizations in maximizing return on investment for their AD projects while reducing the potential for cost overruns, late delivery and scope creep?

Strategic Planning Assumption

To reduce project portfolio disruption and combat the escalating costs of maintaining n-tier and Web applications, by 2002 60 percent of AD organizations will establish a formal release management process encompassing at least two-thirds of their distributed applications portfolio (0.7 probability).

Feedback from our clients indicates that the time and effort spent maintaining distributed n-tier applications — including costs to correct diversions, test and certify, and deploy changes — is growing at alarming rates. In Web, network computer and other client/server environments, a typical AD group's approach to maintenance is inconsistent, harried by unpredictable user demands and the new pressure of so-called "Internet time." Software changes (from small "fixes" to major enhancements) and the release of application variants (e.g., by target platform, location and language) often are erratically managed, "one-off" events, causing confusion and inefficiencies.

A less erratic, more formalized approach, resembling the release management practices of mainframe data-processing organizations, is being adopted by AD organizations as client/server environments mature. *To reduce project portfolio disruption and combat the escalating costs of maintaining n-tier and Web applications, by 2002 60 percent of AD organizations will establish a formal "release management" process encompassing at least two-thirds of their distributed applications portfolio (0.7 probability).* A planned release approach defines a process for: 1) capturing requests for changes; 2) assessing the criticality and impact of such changes; 3) prioritizing the changes, aggregating them into work packages and assigning responsibility; and 4) scheduling releases of the grouped changes. It establishes changes' relative importance and determines which changes should be part of what release. Issues in setting up a release management process revolve around release interval frequency, whether to schedule all changes, and finally, what constitutes a "release" (see Note 1).

The major obstacle to establishing release management practices is persuading users of the benefits of structuring

GartnerGroup

Entire contents © 1998 by Gartner Group, Inc. All rights reserved. Reproduction of this publication in any form without prior written permission is forbidden. The information contained herein has been obtained from sources believed to be reliable. GartnerGroup disclaims all warranties as to the accuracy, completeness or adequacy of such information. GartnerGroup shall have no liability for errors, omissions or inadequacies in the information contained herein or for interpretations thereof. The reader assumes sole responsibility for the selection of these materials to achieve its intended results. The opinions expressed herein are subject to change without notice.

Note 1

Change Package Initiation, Assembly and Approval

Process for Initiation: Informal conversations, E-mails or the like are inadequate to request fixes, changes and enhancements. The process should be structured to collect prioritization information (e.g., costs and risks of delay, or immediate benefits) from those requesting work, since the applications development (AD) group generally cannot accurately weigh the relative business importance of multiple requests.

Exception Process: If most changes circumvent the release management process, it will be quickly marginalized. Nevertheless, persuading business unite (BU) users that responsiveness will be maintained will require flexibility in putting changes into "emergency fix" releases.

Change Package Assembly: Whether work belongs in a major annual release or minor quarterly release, or needs immediate attention, depends on the benefits of quick delivery weighed against the incremental costs of switching resources from other work. Business priorities come first, among them the productivity gains from a defined, cost-efficient process for managing application change.

Release Approval: Traditional production control procedures must be adapted to account for end-user and BU involvement. A "change control board" should approve the promotion of change packages into production, and direct and coordinate orderly desktop updates and user training.

Note 2

Pros and Cons of Scheduled Releases

Advantages

- Savings potential (in terms of resources, time and cost) with respect to:
 - The quality assurance testing function
 - Turnover management function
 - User acceptance test
 - In general, fixed-overhead costs which are associated with introducing any new software
- More stable environment (i.e., modifications will be made less frequently, reducing the opportunity for introducing errors)

Disadvantages

- Possible perception (on the part of the user community) that the IS/AD organization is being "less responsive"
- Possibility that developers and maintenance programmers may become "out-of-date" with the application code because they will be "into the code" less frequently

change on a planned release basis (see Note 2). BU users typically fear responsiveness problems resulting from bundling more changes into fewer releases. Indeed, for mission-critical applications key to the business (i.e., visible to the customer, revenue-producing and used in providing the company's product), batching all changes would be too restrictive, as it would delay time-to-market and limit opportunities for competitive advantage. However, a process for grouping less-important changes into releases can actually better ensure prompt attention to the truly important change/enhancement requests. Other release management benefits — including scheduled training for all affected users, more reliable changes, improvements in application quality and a more stable computing environment — can, once understood by BU management, do much to allay their concerns. The efficiencies and cost reductions achieved through release management are also an important business benefit, but they are generally less compelling to BU managers than to more senior management.

Prior to its turnover into production, the application release must demonstrate production readiness via testing, network certification, appropriate approvals and sufficient documentation, and must show it can meet recommended service levels. A change control board is usually the most effective organizational mechanism to review production readiness. We recommend grouping changes in three categories, and sequencing releases using an easily understood naming convention, like the following:

- *Major Release:* These should be simply numbered (e.g., 3.0) and well-planned, usually in annual or semiannual intervals.
- *Interim Release:* A "3.1" release is also planned — if to a lesser degree — and the work scheduled. Such a release may occur quarterly or bimonthly, but should seldom occur more frequently than monthly.
- *Maintenance Release:* A "3.1.1" release indicates the small, regular (weekly or biweekly) collections of emergency bug fixes and other urgent changes performed on an as-needed basis. Despite most of these changes' shorter-term effort, brief duration and urgency, they should be tracked and collected into "releases." Some changes accepted through the exception process (see Note 1) may result in maintenance releases of some significant new functionality.

Bottom Line: AD organizations should establish a regular, perhaps seasonal rhythm of application releases. Most will find that meeting 70 percent to 80 percent of requests for defect fixes, application changes and enhancements via quarterly minor and semiannual/annual major releases will minimize project portfolio disruption, and will speed delivery of maintenance/enhancement work and, potentially, of new development.

